

Writing device drivers & reverse engineering

Day 2 timetable

TIME	SESSION
09:00-09:30	Introduction and device drivers
09:30-10:00	Bytes, Python and communication protocols + practical
10:00-10:15	Morning coffee
10:15-12:00	Packet captures + practical
12:00-13:00	Lunch
13:00-14:30	Reverse engineering vendor software + practical
14:30-14:45	Afternoon coffee
14:45-15:45	Vendor documentation + practical
15:45-16:00	Questions and wrap-up

Outline

1. What is a driver?
2. Reverse engineering from packet captures
3. Reverse engineering from binaries
4. Using vendor documentation
5. State machines

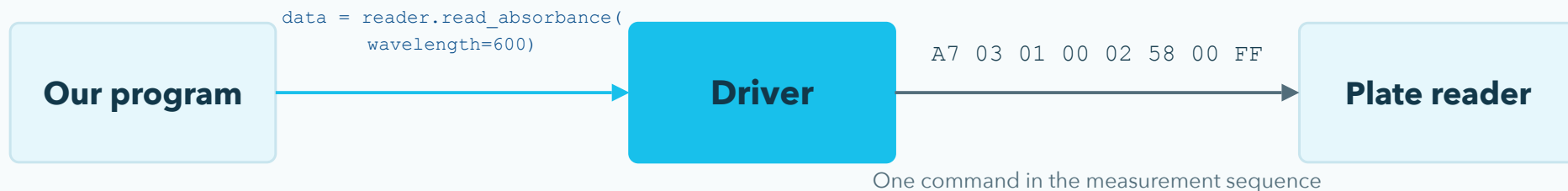
1

What is a driver?

A high-level view

A driver turns intent into device communication

```
data = reader.read_absorbance(wavelength=600)
```



The driver sets up the measurement, sends commands and decodes the result.

The driver is an abstraction

A useful driver hides protocol details while exposing what the device can do.

Without a driver

```
send(bytes.fromhex(
    "A7 03 01 00 02 58 00 FF"
))
send(read_command)
response = receive()
data = decode_absorbance(response)
```

With a driver

```
data = reader.read_absorbance(wavelength=600)
```

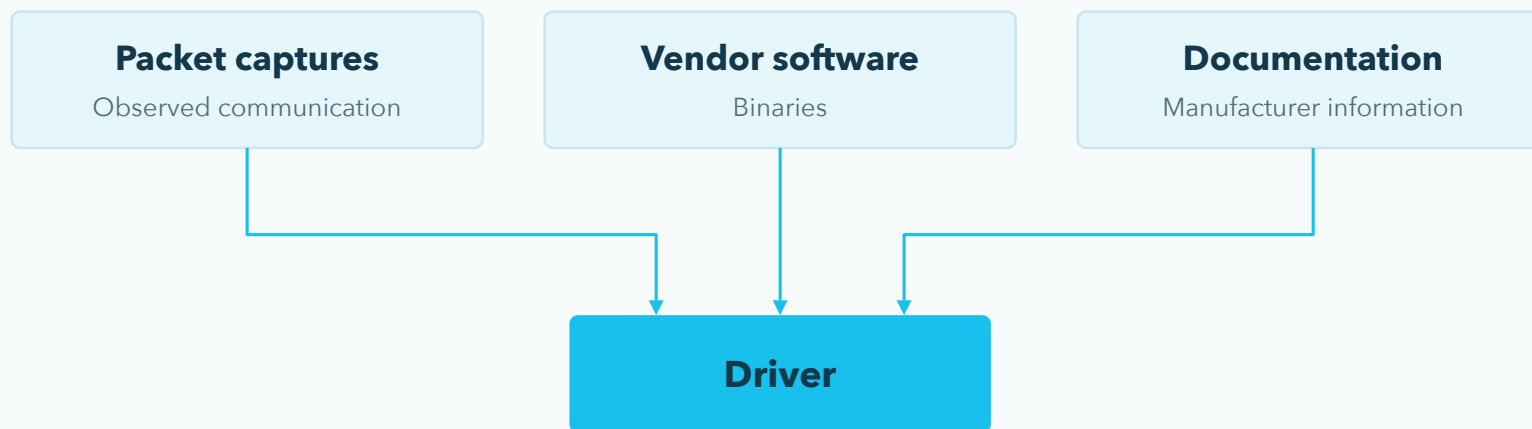
What does the driver actually do?

- Connect to the device
- Turn operations into commands
- Encode parameters
- Send and receive bytes
- Decode responses
- Track device state
- Handle errors

We need to discover how each part works when nobody gives us the code.

Where do drivers come from?

The manufacturer may tell us how the device works. Otherwise, we investigate.



We usually combine more than one source.

2

Reverse engineering from packet captures

Reverse engineering from packet captures

2.1 Background

Data representation, protocols, and transport & hardware layers.

2.2 Analysis

Captures -> comparisons -> hypotheses -> Python

2.3 Practical

Apply the method to the plate reader.

Reverse engineering is like solving a puzzle

Keep this in mind as you work:

- **One finding unlocks another.** A known command can reveal a parameter or a reply.
- **There are tricks.** Comparisons and familiar patterns help you find the next piece.
- **Progress accelerates.** As the pieces connect, the remaining unknowns get easier.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

2.1

Background

2.1 Background

2.1.1

Data representation

Computers need representations

The computer has to represent all of them somehow.

Values

```
600  
"READ"  
0.153  
True
```

Data types

- Integer
- String
- Float
- Boolean

Bits and bytes

A bit

0 or 1

A byte

01000001

8 bits = 1 byte

One byte can represent a value from **0-255**.

Number bases

A base tells us how many different digits we use. These are different ways of representing the same value.

Decimal • base 10

Digits **0-9**.

Place values: ... 100, 10, 1.

$$\begin{aligned} 123 &= 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 \\ &= 100 + 20 + 3 \end{aligned}$$

Binary • base 2

Digits **0 and 1**.

Place values: ... 8, 4, 2, 1.

$$\begin{aligned} 1010_2 &= 1 \times 2^3 + 1 \times 2^1 \\ &= 8 + 2 \\ &= 10_{10} \end{aligned}$$

What is hexadecimal?

Hexadecimal is **base 16**. It uses 16 digits:

0 1 2 3 4 5 6 7 8 9 A B C D E F

A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

Place values: ... 256, 16, 1. The prefix `0x` means hex.

$$0x58 = 5 \times 16 + 8 = 88$$

$$0x0258 = 2 \times 256 + 5 \times 16 + 8 = 600$$

One hex digit = 4 bits. Two hex digits = 1 byte.

`00–FF` in hex = `0–255` in decimal.

Why hexadecimal?

Raw bytes are usually displayed as hex:

```
02 58 41 FF
```

Each pair is **one byte**.

We use hex because it is easier to read than binary.

Bytes do not have meaning by themselves

Meaning depends on how the program interprets the byte.

As a number

Byte `0x41` could mean:

65

As text

The same byte could mean:

"A"

The number 600

As text

"600"

BYTE	CHARACTER
36	6
30	0
30	0

As a binary integer

02 58

$$\begin{aligned} & 0x02 \times 256 + 0x58 \\ &= 2 \times 256 + 88 \\ &= 600 \end{aligned}$$

Strings are bytes too

Text

READ

ASCII / UTF-8

CHARACTER	R	E	A	D
Hex byte	52	45	41	44

52 45 41 44 can represent READ.

ASCII table

ASCII reference

DECIMAL · HEX · CHARACTER

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	16	10	DLE	32	20	SP	48	30	0	64	40	@	80	50	P	96	60	`
1	01	SOH	17	11	DC1	33	21	!	49	31	1	65	41	A	81	51	Q	97	61	a
2	02	STX	18	12	DC2	34	22	"	50	32	2	66	42	B	82	52	R	98	62	b
3	03	ETX	19	13	DC3	35	23	#	51	33	3	67	43	C	83	53	S	99	63	c
4	04	EOT	20	14	DC4	36	24	\$	52	34	4	68	44	D	84	54	T	100	64	d
5	05	ENQ	21	15	NAK	37	25	%	53	35	5	69	45	E	85	55	U	101	65	e
6	06	ACK	22	16	SYN	38	26	&	54	36	6	70	46	F	86	56	V	102	66	f
7	07	BEL	23	17	ETB	39	27	'	55	37	7	71	47	G	87	57	W	103	67	g
8	08	BS	24	18	CAN	40	28	(	56	38	8	72	48	H	88	58	X	104	68	h
9	09	HT	25	19	EM	41	29	)	57	39	9	73	49	I	89	59	Y	105	69	i
10	0A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	J	90	5A	Z	106	6A	j
11	0B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	K	91	5B	[	107	6B	k
12	0C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	L	92	5C	\	108	6C	l
13	0D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	M	93	5D	]	109	6D	m
14	0E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	^	110	6E	n
15	0F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	_	111	6F	o

00-1F: control characters · 20: space (SP) · 21-7E: printable characters · 7F: delete (DEL)

Create and inspect bytes in Python

```
data = bytes.fromhex("02 41 42 ff")

print(data.hex(" ")) # 02 41 42 ff
print(data[0])       # 2: an integer
print(data[:2])      # b'\x02A': bytes
print(list(data))    # [2, 65, 66, 255]
```

A is how Python displays byte
0x41 .

Convert text to bytes

```
text = "READ"
raw = text.encode("ascii")

print(raw)           # b'READ'
print(raw.hex(" "))  # 52 45 41 44
print(raw.decode("ascii")) # READ
```

`b"02 58"` contains five ASCII characters.

`bytes.fromhex("02 58")` contains two bytes.

Pack and unpack with struct

```
import struct

raw = struct.pack(">H", 600)
print(raw.hex(" ")) # 02 58

print(struct.unpack(">H", raw)) # (600,)
number, = struct.unpack(">H", raw)
```

Use `>H` for this device's two-byte whole numbers (0-65535).

This device sends the high byte first. Keep the `>` in the format.

`unpack` returns a tuple, even for one value.

Formats for the workshop

Use these formats in the examples:

FORMAT	TYPE	BYTES
B	Whole number, 0-255	1
>H	Whole number, 0-65535	2
>f	Float	4

```
raw = struct.pack(">BH", 7, 600)
struct.calcsize(">BH")
# 3
struct.unpack(">BH", raw)
# (7, 600)
```

Floating-point values

```
raw = struct.pack(">f", 0.153)
print(raw) # b'>\x1c\xac\x08'
value, = struct.unpack(">f", raw)
print(value)
# 0.15299999713897705
print(value == 0.153) # False

import math
print(math.isclose(value, 0.153, abs_tol=1e-7))
# True
```

Many decimal fractions cannot be stored exactly.

Packing as a 32-bit float rounds the input `0.153` to `0.15299999713897705`.

The decoded value differs from the input, but the difference is less than `1e-7`. With that tolerance, `math.isclose` returns `True`.

Practical: bytes in Python

Open `01_bytes_practical.ipynb`.

- Create, inspect, and slice bytes.
- Convert between text and bytes.
- Pack and unpack values with `struct`.
- Pack and unpack values with the supplied formats.

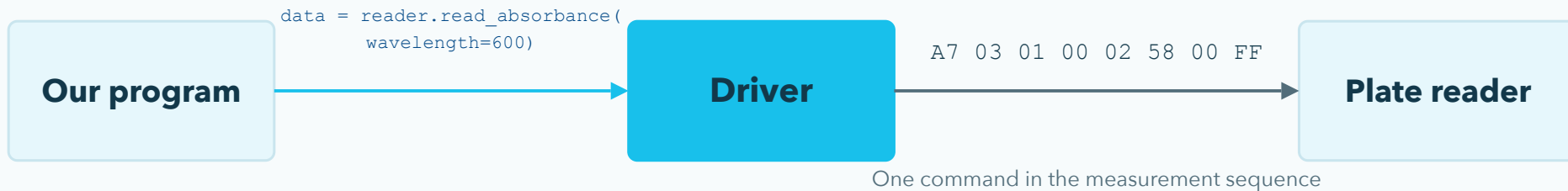
Predict each result, then run the cell.

2.1 Background

2.1.2

wire protocols / "languages"

Device control software sends bytes



The device sends bytes back (more on that later...)

The transport layer differs from device to device: USB, serial, and others.

A protocol

The device and software must agree on the format.

A protocol defines **what those bytes mean**.

For example:

PAYLOAD BYTES

01 02 58

CHECKSUM

5B

Might mean

BYTES

MEANING

01

SET_WAVELENGTH

02 58

600

5B

Checksum

A protocol

Device 1

PAYLOAD BYTES

01 02 58

CHECKSUM

5B

Might mean:

BYTES	MEANING
01	SET_WAVELENGTH
02 58	600
5B	Checksum

Device 2

PAYLOAD BYTES

41 42 53 44 16 00 00

CHECKSUM

30

Might mean:

BYTES	MEANING
"ABS"	READ_ABSORBANCE
44 16 00 00	600.0 nm
30	Checksum

Checksums

Checksums help detect errors. Both sides must use the same rule and the same bytes.

A written letter

"This letter contains 100 words."

The recipient counts them. A different count means a problem.

The same count can still hide changed words.

A protocol message

The sender calculates a value from the bytes and sends it with the message.

The receiver recalculates it. A mismatch means the message failed the check.

Checksums

Example rule: **sum the preceding bytes, modulo 256**. Why 256?

PAYLOAD BYTES

01 02 58

CHECKSUM

5B

$$0x01 + 0x02 + 0x58 = 0x5B$$

The sender appends **5B** as the checksum.

The receiver repeats the calculation and compares the result.

2.1 Background

2.1.3

Transport & hardware layers

Transport: how bytes travel

A connection carries bytes between the computer and device.

Examples: **USB transfers**, a **serial port**, or a **network socket**.

The device protocol defines commands and replies on top of that connection.

| Opening a connection is only the first step: we still need to know what to send.

Hardware and abstraction layers

	LAYER	RESPONSIBILITY
High level 	Python interface	<code>data = reader.read_absorbance(wavelength=600)</code>
	Device protocol	Encode commands and decode replies
	Transport	Send and receive bytes physically
	Low level	Hardware + firmware
		Carry signals and perform the action

Each layer hides details from the layer above it.

2.2

Reverse engineering pcaps

2.2 Analysis

2.2.1

Introduction

When we do not know the protocol

We have

- The device
- Example communication
- Known actions

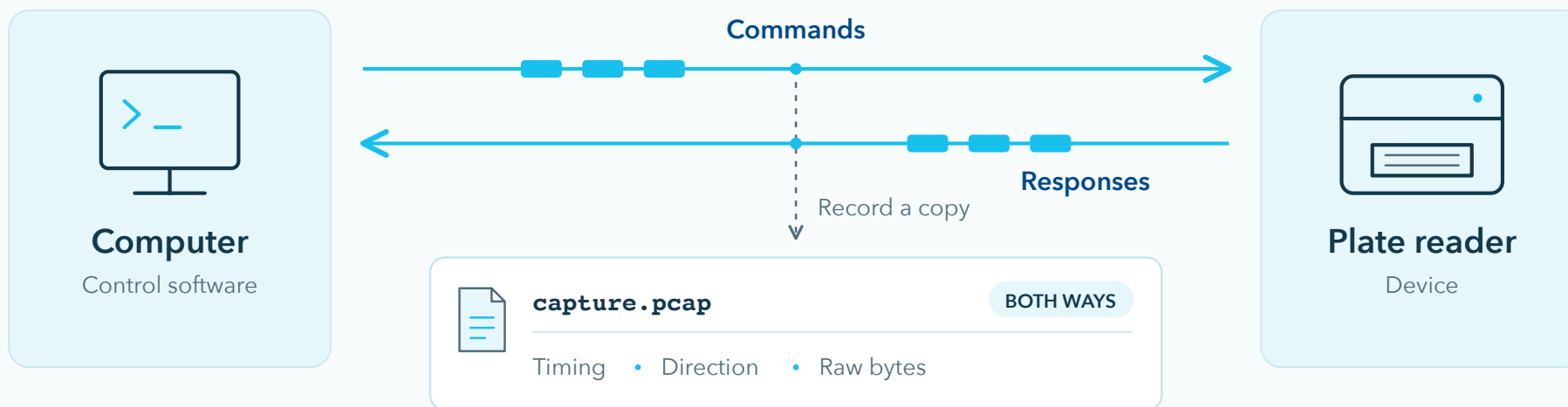
We do not have

- Protocol documentation.

→ **So we infer it.**

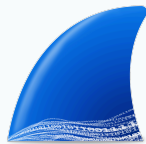
Packet captures

A packet capture (**pcap**) records communication.



(we will use prepared pcaps)

Wireshark



What we can inspect

- Direction
- Timing
- Packet contents
- Raw bytes
- Text representations

The main question

What changed?

Wireshark

The screenshot shows the Wireshark interface with a packet capture file named 'tv-netflix-problems-2011-07-06.pcap'. The packet list on the left shows a DNS response (packet 349) selected. The packet details pane on the right shows the structure of the DNS response, including the transaction ID (0x2188), flags (Standard query response, No error), and the answer section which contains the IP address 192.168.0.1 for the domain cdn-0.nflximg.com.

No.	Time	Source	Destination	Protocol	Length	Info
343	65.142415	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519346 TSecr=551811827
344	65.142715	192.168.0.21	174.129.249.228	HTTP	253	GET /clients/netflix/flash/application.swf?flash_version=flash_lite_2.1&v=1.5&n...
345	65.230738	174.129.249.228	192.168.0.21	TCP	66	80 → 40555 [ACK] Seq=1 Ack=188 Win=6864 Len=0 TSval=551811850 TSecr=491519347
346	65.240742	174.129.249.228	192.168.0.21	HTTP	828	HTTP/1.1 302 Moved Temporarily
347	65.241592	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=188 Ack=763 Win=7424 Len=0 TSval=491519446 TSecr=551811852
348	65.242532	192.168.0.21	192.168.0.1	DNS	77	Standard query 0x2188 A cdn-0.nflximg.com
349	65.276870	192.168.0.1	192.168.0.21	DNS	489	Standard query response 0x2188 A cdn-0.nflximg.com CNAME images.netflix.com.edge...
350	65.277992	192.168.0.21	63.80.242.48	TCP	74	37063 → 80 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=491519482 TSecr=...
351	65.297757	63.80.242.48	192.168.0.21	TCP	74	80 → 37063 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM=1 TSval=3295...
352	65.298396	192.168.0.21	63.80.242.48	TCP	66	37063 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519502 TSecr=3295534130
353	65.298687	192.168.0.21	63.80.242.48	HTTP	153	GET /us/nrd/clients/flash/814540.bun HTTP/1.1
354	65.318730	63.80.242.48	192.168.0.21	TCP	66	80 → 37063 [ACK] Seq=1 Ack=88 Win=5792 Len=0 TSval=3295534151 TSecr=491519503
355	65.321733	63.80.242.48	192.168.0.21	TCP	1514	[TCP segment of a reassembled PDU]

Frame 349: 489 bytes on wire (3912 bits), 489 bytes captured (3912 bits)

- Ethernet II, Src: Globalsc_00:3b:0a (f0:ad:4e:00:3b:0a), Dst: Vizio_14:8a:e1 (00:19:9d:14:8a:e1)
- Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.21
- User Datagram Protocol, Src Port: 53 (53), Dst Port: 34036 (34036)
- Domain Name System (response)
 - [Request In: 348]
 - [Time: 0.034338000 seconds]
 - Transaction ID: 0x2188
 - Flags: 0x8180 Standard query response, No error
 - Questions: 1
 - Answer RRs: 4
 - Authority RRs: 9
 - Additional RRs: 9
 - Queries
 - cdn-0.nflximg.com: type A, class IN
 - Answers
 - Authoritative nameservers

0020 00 15 00 35 84 f4 01 c7 83 3f 21 88 81 80 00 01 ...5.... ?[...]

0030 00 04 00 09 00 09 05 63 64 6e 2d 30 07 6e 66 6cc dn-0.nfl

0040 78 69 6d 67 03 63 6f 6d 00 00 01 00 01 c0 0c 00 ximg.com

0050 05 00 01 00 00 05 29 00 22 06 69 6d 61 67 65 73). ".images

0060 07 6e 65 74 66 6c 69 78 03 63 6f 6d 09 65 64 67 .netflix .com.edg

0070 65 73 75 69 74 65 03 6e 65 74 00 c0 2f 00 05 00 esuite.n et....

Identification of transaction (dns.id), 2 bytes | Packets: 10299 · Displayed: 10299 (100.0%) · Load time: 0:0.182 | Profile: Default

2.2 Analysis

2.2.2

Capture comparison

Controlled comparison

Capture 1

Open the door.

```
A7 03 06 00 00 00 00 FF
```

Capture 2

Close the door.

```
A7 03 07 00 00 00 00 FF
```

Controlled comparison

```
Open:  A7 03 06 00 00 00 00 FF
Close: A7 03 07 00 00 00 00 FF
        ↑
```

Only **one byte** changed: 06 versus 07.

What we changed

Open versus Close.

Hypothesis

Byte at index 2 selects the command: 06 for Open, 07 for Close.

Reverse engineering loop

Change one thing

Observe the communication

Compare captures

Form a hypothesis

Test it



2.2 Analysis

2.2.3

Tips and tricks for analyzing pcaps

#1 Repeated traffic often means queries

Repeated requests while idle? Probably **status queries**.

```
> A7 03 02 00 00 00 00 FF  
> A7 03 02 00 00 00 00 FF  
> A7 03 02 00 00 00 00 FF
```

Requests shown; replies omitted.

Change the device state. Compare the replies.

Could also be keepalives or retries.

#2 Short replies can signal errors

Expected measurement data, got a short reply?

- Compare the header, command ID, and ending.
- Look for a status byte that changed.
- Check for rejection, missing bytes, or no response.

`read(198)` can return fewer than 198 bytes.

#3 Earlier commands may prepare the device

Look before the action for **door commands or measurement settings**.

1. Start from a known state.
2. Add one suspected setup command.
3. Retry the same request.

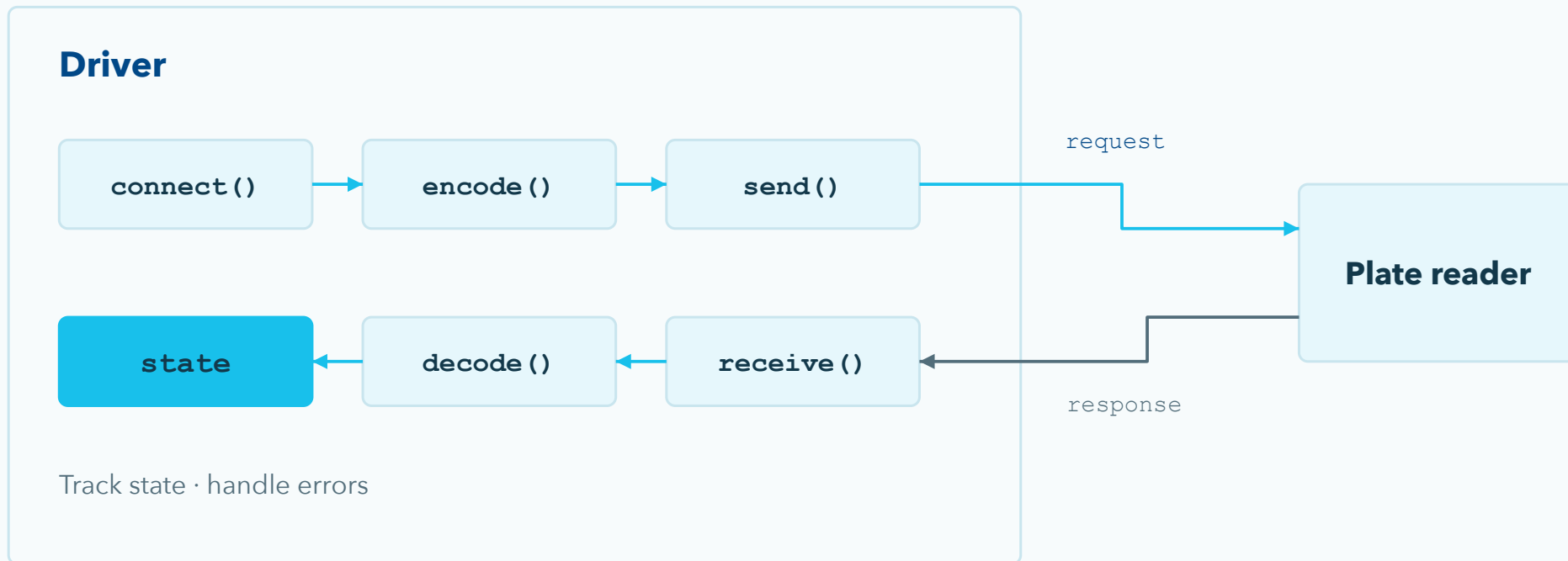
Reconnecting may preserve state. Reset the device when repeating the experiment.

2.2 Analysis

2.2.4

Requests & responses

Requests and responses



Decode response bytes: boolean

Boolean: is the door open?

```
response = bytes.fromhex("A7 83 02 00 00 00 00 01 00 01 00 FF")
is_open = response[9] == 1 # 00 = closed, 01 = open
print(is_open) # True
```

`response[9]` selects the door-status byte. The protocol defines `00` as closed and `01` as open.

Decode response bytes: integer

Integer: accepted wavelength

```
import struct

response = bytes.fromhex("A7 83 01 00 02 58 00 FF")
wavelength, = struct.unpack(">H", response[4:6]) # two bytes: 02 58
print(wavelength) # 600
```

The protocol determines each field's position and size. Use `>H` to read this device's two-byte whole numbers.

2.3

Reverse engineering pcaps practical

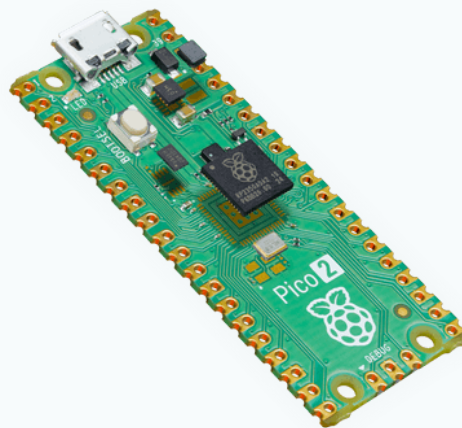
What are we reverse engineering?

- A Raspberry Pi Pico 2
- Pretending to be a plate reader
- We know what it does
- We do not know how to talk to it

Our goal

Python -> device -> response

I'm a plate reader!



What's in the zip

Start with `02_door_practical.ipynb` and `captures/door.txt`.

Notebooks

```
01_bytes_practical.ipynb
02_door_practical.ipynb
03_reading_practical.ipynb
04_vendor_software_practical.ipynb
```

Communication transcripts

```
captures/door.txt
captures/reading.txt
captures/absorbance.txt
```

Open these text files beside the notebook.
> marks requests; < marks responses.

Your task

All practicals build on the same API. Each one gives you more tools to figure out how the device works: captures, vendor software, then documentation.

You can build the full driver from packet captures alone. For now, aim for a good start. Don't be discouraged if you don't finish: we'll keep building on your work in the next practicals.

Using the provided captures:

1. Identify commands, parameters, and responses.
2. Work out the message format.
3. Reproduce it in Python.
4. Build an interface to the plate reader.

3

Reverse engineering from binaries

Reverse engineering from binaries

3.1 Background

Compiled programs, C# & .NET, and decompilation tools.

3.2 Investigation

Find clues, decode commands and responses, and verify against captures.

3.3 Practical

Use the decompiled application to build the plate-reader interface in Python.

3.1

Background

3.1 Background

3.1.1

Compiled programs

Another source of clues: the software

So far, we have treated the device and its protocol as a black box.

We can observe:

```
Program -> bytes -> device
```

If we have access to the program controlling the device, can we figure out what the program is doing?

This is software reverse engineering.

Compiled programs

How much can we recover from the application?

When we write Python

```
reader.read_plate()
```

We are used to having the source code right in front of us.

Many applications

Source code -> compiler -> application

The user gets the application, but usually not the original source code.

3.1 Background

3.1.2

C# & .NET

C# and .NET

C# is used to build desktop applications, services, instrument software, and other programs.

```
int wavelength = 450;
string command = "READ";
Console.WriteLine(command);
```

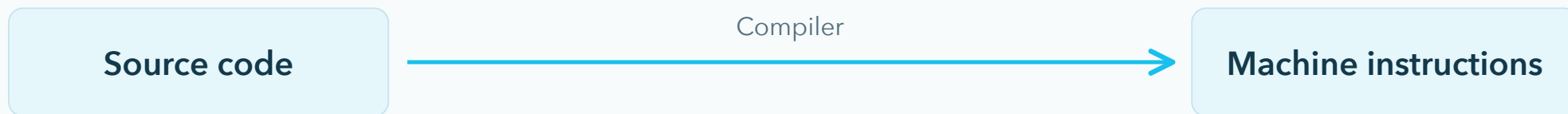
Familiar concepts

C#	MEANING
int	Integer
string	Text
byte	One byte
byte[]	Sequence of bytes

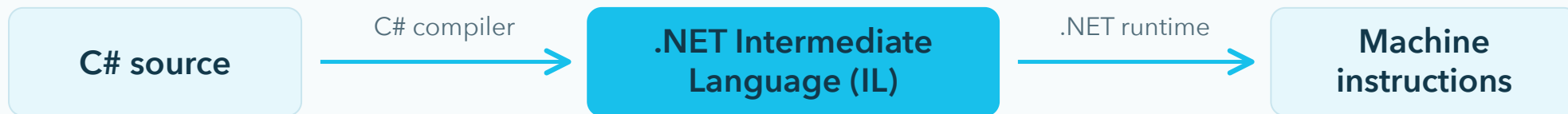
You do not need to learn C# for this exercise.

C# compilation is interesting

A simplified view



.NET adds a layer



IL retains useful information that helps us reverse engineer the application.

What is inside a .NET program?

A compiled `.exe` or `.dll` can contain:

- Classes and methods
- Method parameters and data types
- Strings and constants
- .NET instructions

For example, the original program might contain:

```
public void SetWavelength(int wavelength)
{
    // ...
}
```

Useful information about that method can survive compilation.

That gives us more to work with than an opaque stream of machine instructions.

3.1 Background

3.1.3

Decompilation & tools

Decompilation

A **disassembler** shows instructions.

A **decompiler** tries to reconstruct readable source code.

```
program.exe  
  ↓ decompiler  
C#-like source code
```

A reconstruction

Decompiled code is not necessarily the original source code.

It reconstructs code that behaves like the compiled program.

Comments, formatting, some names, and higher-level structures may be gone or changed.

Tool #1: ILSpy

ILSpy is an open-source .NET assembly browser and decompiler.

Give it a .NET executable or library and explore its contents much like navigating source code.

Hunt for the parts that matter.

```
PlateReader.exe
├── Device
│   ├── Connect()
│   ├── SetWavelength()
│   └── ReadPlate()
├── Protocol
│   ├── BuildCommand()
│   └── ParseResponse()
└── Utilities
```

Tool #2: dotPeek

dotPeek, from JetBrains, performs a similar job.

Both tools let us inspect compiled .NET applications and reconstruct C#.

For this workshop, the exact tool is less important than the idea:

```
Compiled .NET program -> decompiler -> code we can investigate
```

3.2

Investigation

3.2 Investigation

3.2.1

Finding clues

Finding the relevant code

A real application might contain hundreds of thousands of lines of code.

Reverse engineering often means finding a needle in a haystack.

Search for clues

```
plate, read, wavelength, absorbance,  
luminescence, serial, command, packet,  
byte
```

Also search for strings you have already observed elsewhere.

Strings are excellent clues

Trace from something recognizable toward the protocol.

Start with the UI

"Set wavelength"

"Reading plate..."

"Invalid response from device"

Search for a string. You might land in:

```
private void ReadPlateButton_Click(...)
{
    reader.ReadPlate(...);
}
```

Follow the call

Now follow `ReadPlate()`:

```
public byte[] ReadPlate(int wavelength)
{
    return SendCommand(...);
}
```

Then follow `SendCommand()`.

The boundary between concepts and bytes

Our main target is often where software turns concepts into bytes.

```
"read absorbance at 450 nm"
```

```
-> C# methods -> byte array -> USB / serial -> device
```

If we find the function that builds the byte array, we may be looking directly at the **protocol implementation**.

3.2 Investigation

3.2.2

Command encoding

Suppose we find this

```
byte[] BuildCommand(Command command, ushort argument = 0)
{
    byte[] packet = new byte[8];
    packet[0] = 0xa7;
    packet[1] = 0x03;
    packet[2] = (byte)command;
    packet[3] = 0;
    packet[4] = (byte)(argument >> 8);
    packet[5] = (byte)(argument & 0xff);
    packet[6] = 0;
    packet[7] = 0xff;
    return packet;
}
```

What can we learn?

An **eight-byte packet**.

Byte **2** selects the command.

Bytes **4–5** hold its argument, high byte first.

```
BuildCommand(
    Command.SetWavelength, 450);
```

Integers have to become bytes

Use `struct.pack(">H", 450)` to create the two argument bytes.

Wavelength = 450

In hexadecimal:

```
450 = 0x01C2
```

Our communication channel sends bytes, so the program splits the number.

The packet

High byte = 01

Low byte = C2

Sent: A7 03 01 00 01 C2 00 FF

Now compare it to the capture

Two independent views of the same system. Together, they are much more powerful.

Packet capture

```
A7 03 01 00 01 C2 00 FF
```

The capture tells us **what actually happened**.

Decompiled program

```
// SetWavelength = 0x01, argument = 450  
packet[2] = (byte)command;  
packet[4] = (byte)(argument >> 8);  
packet[5] = (byte)(argument & 0xff);
```

The software suggests **why those bytes were sent**.

3.2 Investigation

3.2.3

Response decoding

Responses work the same way

```
ushort[] readings =  
    PacketCodec.ReadArray(response, 4, 96);
```

Another clue

Start at byte index 4.

Read 96 values, one per well.

`ReadArray` takes two bytes per value. In Python, use `>H` for each pair.

3.2 Investigation

3.2.4

Verification

Names can give the game away

Sometimes you get lucky.

```
const byte CMD_READ_LUMINESCENCE = 0x20;  
const byte CMD_READ_ABSORBANCE   = 0x21;  
const byte CMD_SET_GAIN           = 0x31;
```

Now we have a stronger hypothesis than “byte 1 changes during absorbance measurements”:

0x21 -> READ_ABSORBANCE

Then we verify it against observed traffic.

Cross-check the decompiled code

You might encounter

- Unused code
- Old protocol implementations
- Debug functionality
- Multiple device versions
- Compiler-generated code
- Obfuscated code

The decompiler reconstructs higher-level C# from compiled instructions.

Build confidence

Decompiler + packet capture + experiments

Cross-check your conclusions.

Our reverse-engineering toolbox

Observe

Run the software -> capture communication -> compare experiments

Inspect

Open the software -> search / decompile -> find protocol code

Verify

Form a hypothesis -> send our own bytes -> observe the device

No single technique has to give us the whole answer.

The goal

From

Mystery application ↔ mystery protocol ↔ device



To

Our Python code ↔ **protocol we understand** ↔ **device**

Once we understand the interface well enough, we no longer need the original software to communicate with the device.

3.3

Practical: Decode the vendor protocol

Extend your driver using vendor code

You have

A decompiled .NET application, prepared packet captures, a “plate reader,” and a Jupyter Notebook.

Determine

- What commands exist?
- What bytes identify each command?
- How are parameters encoded?
- How are responses encoded?
- What data types are involved?

Continue your driver

Build on the packet-capture practical and keep the same API. Use the vendor code to fill in what you could not infer from captures:

```
data = reader.read_absorbance(  
    wavelength=600  
)
```

4

Using vendor documentation

Sometimes someone actually tells you

Manufacturers may publish:

- Communication protocols and command tables
- Programmer's manuals
- SDK documentation
- Service manuals
- Firmware documentation

Look for documentation before reverse engineering a device.

What we're looking for

Transport

USB, serial or TCP. Baud rate, VID:PID and endpoints.

Commands

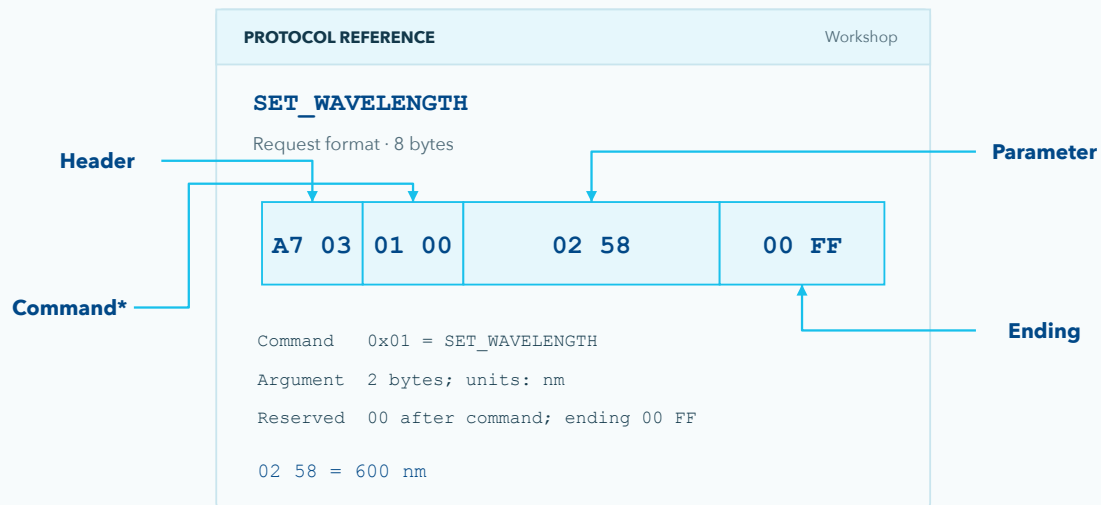
Command IDs, request formats and parameters.

Responses

Packet formats, data types and error codes.

Behavior

Valid command order, timing and device states.



These details connect the bytes to the operations we want to implement.

Documentation is evidence too

Documentation may be incomplete, outdated, ambiguous or wrong. It may describe another firmware version.

Documentation tells us what should happen.

Captures tell us what did happen.

Experiments tell us what the device accepts.

Turning documentation into code

The command table supplies the ID and parameter encoding.

Example manual

```
Command:  SET_WAVELENGTH
ID:        0x01
Argument:  wavelength
Size:      2 bytes, high byte first
```

600 encodes as `02 58`.

Python

```
def set_wavelength(nm):
    reply = send_command(0x01, nm)
    return require_success(reply)
```

`send_command` packs the argument with `struct.pack(">H", argument)`, sends the command and receives its reply.

What documentation may leave out

- It might be wrong.
- It might be incomplete.
- It might describe individual commands without explaining how they interact.

Knowing the commands does not tell us which sequences are valid or how one command changes what can happen next.

Use captures, vendor code and device experiments to fill the gaps and check the claims.

More on command interactions in the next section: State machines.

4.1

Documentation practical

Combine the manual with captures and experiments to build a working reading sequence.

Practical: build the reading sequence

Open `05_firmware_reference_practical.ipynb`.

```
data = read_absorbance(wavelength=600)
```

All individual command functions and both decoders are supplied, including `decode_absorbance()`.

Your task: choose the calls and their order to implement `read_absorbance()`.

Use the manual, earlier captures and device experiments. You can start here even if the earlier practicals are unfinished.

Practical: find both mistakes in the docs

The manual contains **two deliberate mistakes**. Finding both is part of this practical.

1. Compare the manual with your captures and the vendor code.
2. Test the behavior on the device.
3. Identify both incorrect claims and support each correction with evidence.
4. Use your findings to finish the reading sequence.

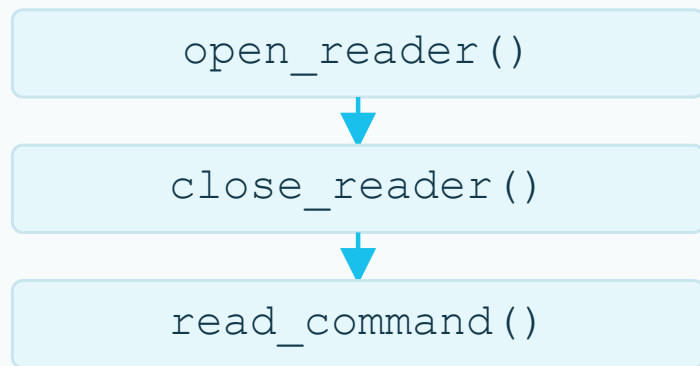
Having the docs still leaves work: combining commands and checking their behavior requires evidence from other sources.

5

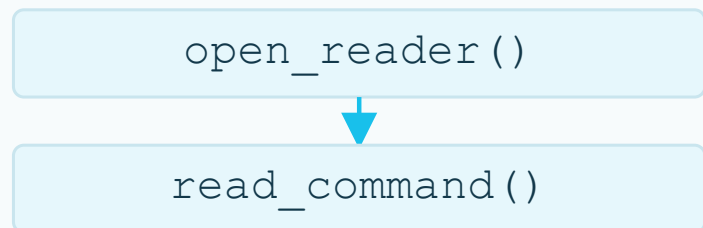
State machines

Commands aren't independent

Both sequences explicitly open the reader first.



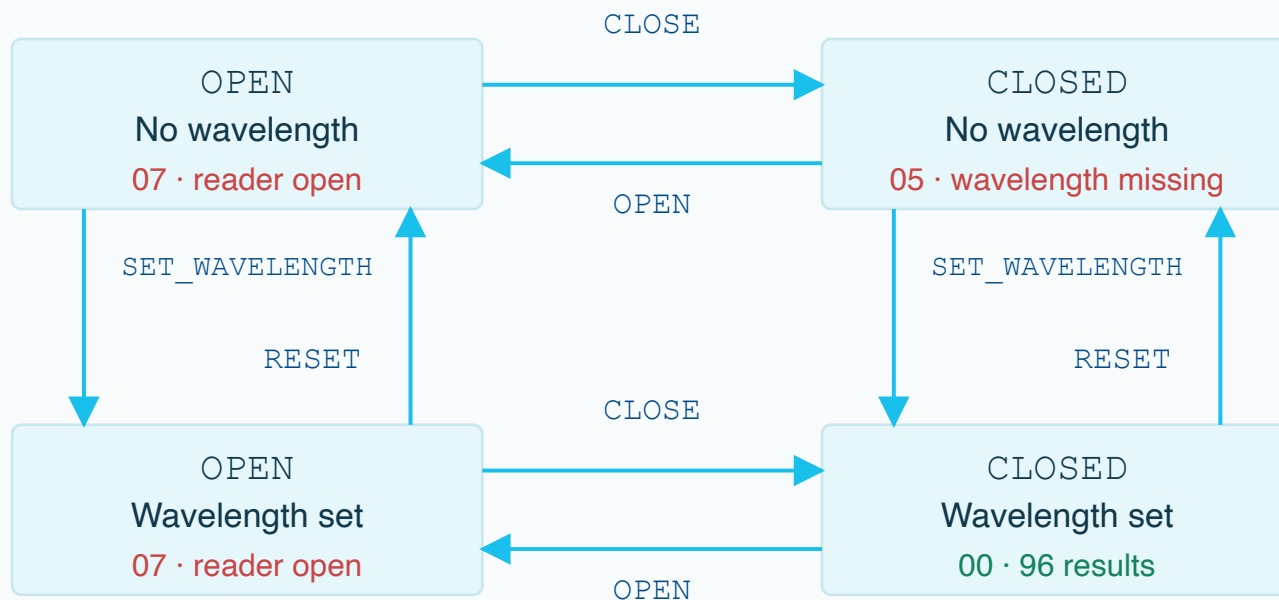
00 · 96 results



07 · reader open

Here, `read_command()` sends `READ_LUMINESCENCE` and receives its reply.

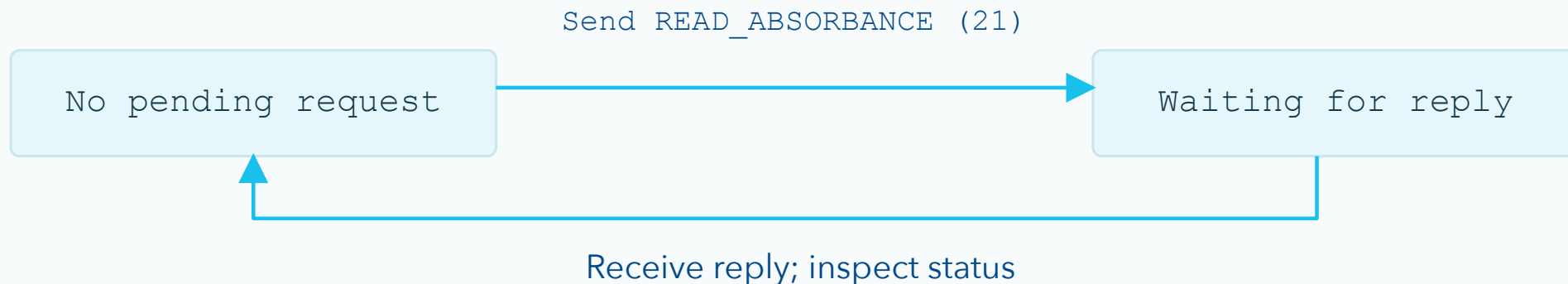
The reader's state



Each node shows the **absorbance read** outcome. Reading leaves the state unchanged.

The driver also has state

`send_command()` sends a request, then waits for the reply before returning.



Check the status before decoding: absorbance returns 390 bytes on success or 8 bytes on error.

Reconstructing state from observations

Start closed, with no wavelength set. Change one thing at a time.

CALL		OBSERVATION
<code>read_absorbance_command()</code>		05 · wavelength missing
<code>set_wavelength(600)</code>		00 · wavelength accepted
<code>read_absorbance_command()</code>		00 · 96 results
<code>read_absorbance_command()</code>		00 · same 96 results

The successful repeat shows that wavelength persists. `request_state()` lets us inspect it directly.

Conclusion

From bytes to a working driver, one verified step at a time.

- **Combine evidence:** packet captures, vendor code and documentation each reveal part of the protocol.
- **Test your hypotheses:** change one thing, predict the reply, then check it on the device.
- **Account for state:** the right command can fail if the device is not ready for it.
- **Build a useful interface:** encode commands, check replies and turn raw bytes into meaningful results.